



FULL STACK WEB DEVELOPMENT LAB

Ms. Fiza Asmat

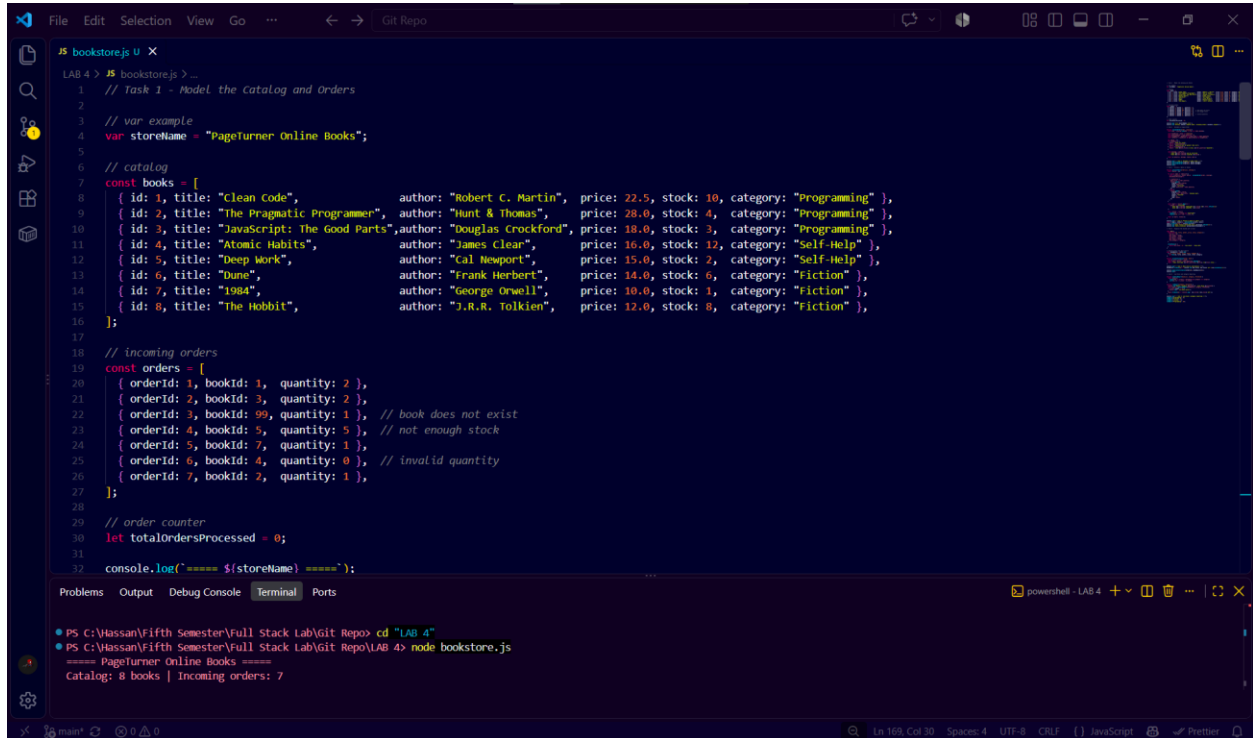


HASSAN SHAHID
241790
BSCS 5C

LAB 04

Home Task:

Task 1 — Model the Catalog and Orders (Variables, Objects, Arrays):



```
1 // Task 1 - Model the Catalog and Orders
2
3 // var example
4 var storeName = "PageTurner Online Books";
5
6 // catalog
7 const books = [
8   { id: 1, title: "Clean Code", author: "Robert C. Martin", price: 22.5, stock: 10, category: "Programming" },
9   { id: 2, title: "The Pragmatic Programmer", author: "Hunt & Thomas", price: 28.0, stock: 4, category: "Programming" },
10  { id: 3, title: "JavaScript: The Good Parts", author: "Douglas Crockford", price: 18.0, stock: 3, category: "Programming" },
11  { id: 4, title: "Atomic Habits", author: "James Clear", price: 16.0, stock: 12, category: "Self-Help" },
12  { id: 5, title: "Deep Work", author: "Cal Newport", price: 15.0, stock: 2, category: "Self-Help" },
13  { id: 6, title: "Dune", author: "Frank Herbert", price: 14.0, stock: 6, category: "Fiction" },
14  { id: 7, title: "1984", author: "George Orwell", price: 10.0, stock: 1, category: "Fiction" },
15  { id: 8, title: "The Hobbit", author: "J.R.R. Tolkien", price: 12.0, stock: 8, category: "Fiction" },
16 ];
17
18 // incoming orders
19 const orders = [
20   { orderId: 1, bookId: 1, quantity: 2 },
21   { orderId: 2, bookId: 3, quantity: 2 },
22   { orderId: 3, bookId: 99, quantity: 1 }, // book does not exist
23   { orderId: 4, bookId: 5, quantity: 5 }, // not enough stock
24   { orderId: 5, bookId: 7, quantity: 1 },
25   { orderId: 6, bookId: 4, quantity: 0 }, // invalid quantity
26   { orderId: 7, bookId: 2, quantity: 1 },
27 ];
28
29 // order counter
30 let totalOrdersProcessed = 0;
31
32 console.log(`===== ${storeName} =====`);
```

Problems Output Debug Console Terminal Ports

```
PS C:\Vassan\Fifth Semester\Full Stack Lab\git Repo> cd "LAB 4"
PS C:\Vassan\Fifth Semester\Full Stack Lab\git Repo\LAB 4> node bookstore.js
===== PageTurner Online Books =====
Catalog: 8 books | Incoming orders: 7
```

Task 2 — Validate a Single Order (Conditions, Operators, Ternary Operator):



```
1 // Task 2 - Validate a Single Order
2
3 function validateOrder(order, catalog) {
4   const book = catalog.find((b) => b.id === order.bookId);
5
6   const bookExists = book !== undefined;
7   const quantityValid = order.quantity > 0;
8   const enoughStock = bookExists && book.stock >= order.quantity;
9   const canFulfill = bookExists && quantityValid && enoughStock;
10
11   let reason = "";
12   if (!bookExists) {
13     reason = "book not found";
14   } else if (!quantityValid) {
15     reason = "quantity must be greater than zero";
16   } else if (!enoughStock) {
17     reason = `only ${book.stock} in stock, ${order.quantity} requested`;
18   }
19
20   const message = canFulfill
21     ? `Order #${order.orderId} can be fulfilled.`
22     : `Order #${order.orderId} rejected: ${reason}`;
23
24   return { canFulfill, message, reason, book };
25 }
26
27 console.log("--- Task 2: Validate a single order ---");
28 console.log(validateOrder(orders[0], books).message);
29 console.log(validateOrder(orders[2], books).message);
30 console.log();
```

```
Problems Output Debug Console Terminal Ports
● PS C:\Vassan\Fifth Semester\Full Stack Lab\Git Repo> cd "LAB 4"
● PS C:\Vassan\Fifth Semester\Full Stack Lab\Git Repo\LAB 4> node bookstore.js

===== PageTurner Online Books =====
Catalog: 8 books | Incoming orders: 7

--- Task 2: Validate a single order ---
Order #1 can be fulfilled.
Order #3 rejected: book not found.
```

Task 3 — Process a Batch of Orders (Loops, Array Methods):

```
66 // Task 3 - Process a Batch of Orders
67
68 function processOrders(orderList, catalog) {
69   const results = [];
70
71   for (const order of orderList) {
72     const { canFulfill, reason, book } = validateOrder(order, catalog);
73     totalOrdersProcessed++;
74
75     if (canFulfill) {
76       book.stock -= order.quantity;
77       results.push({
78         orderId: order.orderId,
79         title: book.title,
80         status: "fulfilled",
81         total: book.price * order.quantity,
82       });
83     } else {
84       results.push({
85         orderId: order.orderId,
86         title: book ? book.title : "Unknown book",
87         status: "rejected",
88         total: 0,
89         note: reason,
90       });
91     }
92   }
93
94   const report = results.map((r) => {
95     r.status === "fulfilled"
96       ? `Order #${r.orderId}: FULFILLED - ${r.title} (${r.total.toFixed(2)})`
97       : `Order #${r.orderId}: REJECTED - ${r.note}`
98   });
99
100   const revenue = results
101     .filter((r) => r.status === "fulfilled")
102     .reduce((sum, r) => sum + r.total, 0);
103
104   return { report, revenue };
105 }
106
107 console.log("--- Task 3: Process a batch of orders ---");
108 const { report, revenue } = processOrders(orders, books);
109 report.forEach((line) => console.log(line));
110 console.log(`Total revenue (fulfilled only): ${revenue.toFixed(2)}`);
111 console.log(`Orders processed: ${totalOrdersProcessed}\n`);
```

```
--- Task 3: Process a batch of orders ---
Order #1: FULFILLED - Clean Code ($45.00)
Order #2: FULFILLED - JavaScript: The Good Parts ($36.00)
Order #3: REJECTED - book not found
Order #4: REJECTED - only 2 in stock, 5 requested
Order #5: FULFILLED - 1984 ($10.00)
Order #6: REJECTED - quantity must be greater than zero
Order #7: FULFILLED - The Pragmatic Programmer ($28.00)
Total revenue (fulfilled only): $119.00
Orders processed: 7
```

Task 4 — Organize the System with a Class (ES6 Classes, Destructuring, Template Literals):

```
113 // Task 4 - Organize the System with a Class
114
115 class Book {
116   constructor(id, title, author, price, stock, category) {
117     this.id = id;
118     this.title = title;
119     this.author = author;
120     this.price = price;
121     this.stock = stock;
122     this.category = category;
123   }
124
125   stockStatus() {
126     return this.stock < 5 ? "Low stock" : "Stock OK";
127   }
128 }
129
130 // Convert books into Book objects
131 const bookObjects = books.map(
132   ({ id, title, author, price, stock, category }) =>
133     new Book(id, title, author, price, stock, category)
134 );
135
136 function printConfirmation(order, book) {
137   const { title, price } = book;
138   const total = (price * order.quantity).toFixed(2);
139   return `Order confirmed: ${order.quantity} x ${title} - ${total} total.`;
140 }
141
142 console.log("--- Task 4: Book class & confirmation ---");
143 bookObjects.forEach((b) => console.log(`${b.title}: ${b.stock} left + ${b.stockStatus()}`));
144
145 console.log(printConfirmation(orders[0], bookObjects[0]));
146 console.log();
```

```
--- Task 4: Book class & confirmation ---  
Clean Code: 8 left + Stock OK  
The Pragmatic Programmer: 3 left + Low stock  
JavaScript: The Good Parts: 1 left + Low stock  
Atomic Habits: 12 left + Stock OK  
Deep Work: 2 left + Low stock  
Dune: 6 left + Stock OK  
1984: 0 left + Low stock  
The Hobbit: 8 left + Stock OK  
Order confirmed: 2 x Clean Code - $45.00 total.
```

Task 5 — Low-Stock and Category Reporting (Combined Techniques):

```
148 // Task 5 - Low-Stock and Category Reporting  
149  
150 function lowStockReport(bookList, category, threshold) {  
151   return bookList  
152     .filter((b) => b.stock < threshold && b.category === category)  
153     .sort((a, b) => a.stock - b.stock);  
154 }  
155  
156 function runReport(category, threshold) {  
157   console.log(`Report: category "${category}", stock below ${threshold}`);  
158   const result = lowStockReport(bookObjects, category, threshold);  
159   if (result.length === 0) {  
160     console.log(" (no books match)");  
161   }  
162   result.forEach((b) => console.log(` ${b.title} - ${b.stock} left`));  
163 }  
164  
165 console.log("--- Task 5: Low-stock & category reporting ---");  
166 runReport("Programming", 5);  
167 runReport("Fiction", 5);  
168 runReport("Fiction", 7);  
169 runReport("Programming", 10);
```

```
--- Task 5: Low-stock & category reporting ---  
Report: category "Programming", stock below 5  
JavaScript: The Good Parts - 1 left  
The Pragmatic Programmer - 3 left  
Report: category "Fiction", stock below 5  
1984 - 0 left  
Report: category "Fiction", stock below 7  
1984 - 0 left  
Dune - 6 left  
Report: category "Programming", stock below 10  
JavaScript: The Good Parts - 1 left  
The Pragmatic Programmer - 3 left  
Clean Code - 8 left  
PS C:\Hassan\Fifth Semester\Full Stack Lab\Git Repo\LAB 4>
```